BulSUScholar Algorithm Metrics Audit

[T] Execution date: 2026-10-28T04:28:01.667642+00:00 (UTC). Existing application code, source fixtures, manuscripts, historical evidence and database records were not changed. The audit executed only synthetic, offline tests and ephemeral local SQL functions. All 437 tracked file hashes remained unchanged.

[T] Main finding: the existing combined backend pair classifier reproduced TP=5, FP=1, FN=0, TN=4 on ten pre-existing synthetic labeled pairs. Precision=83.33%, Recall=100.00%, audit-derived F1=90.91%, Accuracy=90.00%. This is a limited fixture evaluation, not independently verified student-population accuracy, award-conflict accuracy, or proof that every duplicate is detected.

[T] Separate evidence: 48 matching/roster backend regression tests and 33 document-related tests passed. Seven current SQL profile-matching assertions passed in ephemeral PGlite. The document rerun reproduced COR16/16 and ROG9/9 present-field comparisons across three synthetic text PDFs, while retaining four validation limitations and two dependency-blocked OCR cases. None of these test pass counts is a confusion-matrix numerator.

[S] Critical separation: scalar Levenshtein similarity, the backend duplicate-pair classifier, admin grouping, signup roster-name checking, browser ROG diagnostics, exact import-profile matching, and scholarship commitment guards are different mechanisms. They must not be evaluated as interchangeable stages or have one result copied to every algorithm name.

Finding labels used throughout: [S] Verified in source code; [T] Verified through test execution; [P] Proposed evaluation methodology; [N] Not implemented in the inspected mechanism; [U] Not verifiable from available evidence. A source feature does not imply verified deployment. A synthetic fixture label is not independent verification of an actual student.

1. Verified Algorithms and Source Code Locations

[S] Custom Levenshtein dynamic programming appears in backend/grantor_algorithms.py:32-56 (levenshtein_similarity) and backend/signup_service.py:485-508 (levenshtein_similarity). Browser implementations appear in src/pages/SignupPage.jsx:486-510 (getLevenshteinDistance); src/pages/SignupPage.jsx:512-518 (getLevenshteinSimilarity) and src/services/grantorService.js:362-386 (levenshteinSimilarity). These are custom loops, not a Levenshtein package call. Strings are normalized before comparison.

[S] The backend grantor normalizer lowercases, collapses whitespace and replaces non-ASCII alphanumeric runs with spaces. Identifiers then remove whitespace; phone strings retain digits only. The JS normalizer first strips combining accents with Unicode NFD, so it is not identical to Python for accented names. Email punctuation is removed, potentially collapsing distinct addresses; local 09 vs +63 phone equivalence is not implemented in the pair helper.

[S] The backend pair evaluator compares available Student ID, full name, email, contact number, course, year level, city and province. Name similarity takes the maximum of direct order and sorted tokens. It computes similarities even when exact IDs match; there is no universal ID-first/fuzzy-only-if-missing execution sequence. Source: backend/grantor_algorithms.py:68-123 (evaluate_scholar_duplicate).

[S] find_scholar_duplicate scans supplied records and returns the flagged candidate with highest weighted score; ties retain the first encountered. There is no ambiguous-match tier. It does not itself query all awards or filter active scholarship status. Admin grouping adds its own score-only condition and is greedy/order-dependent. Sources: backend/grantor_algorithms.py:126-148 (find_scholar_duplicate); backend/grantor_algorithms.py:206-262 (check_student_table_duplicates).

[S] Additional lookup helpers implement exact ID or normalized name-parts/address matching; they are not weighted Levenshtein decisions. Shared address fields must agree, and missing middle initials can be tolerated. The admin fallback chooses a first matching student. Sources: backend/grantor_algorithms.py:326-333 (matches_grantor_scholar_to_student); backend/grantor_algorithms.py:366-396 (match_admin_grantor_students).

[S] Signup roster-name checking uses direct/sorted Levenshtein or same-last-token overlap, after roster queries using the supplied ID. Missing names pass the name helper. These results are not official scholarship authorization. Sources: backend/signup_service.py:549-562 (is_similar_roster_name); backend/signup_service.py:575-612 (find_roster_identity_conflicts).

[S] Browser validateCogIdentity calculates .75 exact-ID + .25 name similarity, with .85 score/.70 name gates when both names exist. Name similarity is the larger of sorted Levenshtein and token overlap. However, the current scanUploadedDocument caller only logs a failed check as inconclusive and continues to server validation. The comparison is not a server-enforced ROG-owner safeguard. Sources: src/pages/SignupPage.jsx:585-697 (validateCogIdentity); src/pages/SignupPage.jsx:981-1093 (scanUploadedDocument).

[S] Current import identity has changed since the older October 23 documentation: when both Student IDs exist, compare them exactly; when either is missing, compare an exact normalized name/course/year profile. Comma-order names, course aliases and numeric/ordinal years are canonicalized. Distinct explicit IDs remain distinct even if profiles coincide. Name-only imports remain unlinked and cannot authorize an account award. Sources: backend/roster_workflow_service.py:118-138 (_roster_profile_key); backend/roster_workflow_service.py:141-146 (_same_roster_identity); supabase/migrations/20261026043502_manual_roster_name_and_course_matching.sql:3-32 (roster_import_profile_key); supabase/migrations/20261026034945_cross_grantor_roster_identity_guard.sql:16-24 (roster_import_matches).

[S] Official awards remain a separate transaction with stored exact ID/name, active account/program/provider, cycle and commitment guards. Current-cycle Completed/Finished awards remain recognized commitments; explicit older cycles do not independently block a new cycle. Cancelled/Terminated/Inactive rows are excluded from active checks, but another commitment can still block. Unknown-cycle legacy awards are handled conservatively. Sources: supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:69-194 (assign_authoritative_roster_scholarship); supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:26-32 (roster_record_active); supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:34-42 (roster_application_awarded); supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:447-474 (roster_application_blocker).

[S] Full Admin review of official roster conflicts requires role full_admin, an accepted resolution action and a 10-2000-character reason. A fuzzy pair flag does not automatically create an official roster conflict or determine an award. Source: backend/roster_workflow_service.py:520-542 (resolve_roster_conflict). [U] Independent real-student identity/conflict adjudication, production migration state and current deployment were not verified.

[S] COR/ROG extraction uses pdfplumber text/layout, regex/title/field rules, PDF table/word grade parsing and an optional local Tesseract OCR fallback. It does not calculate Levenshtein confidence inside document_scanner. A browser identity diagnostic and signup roster-name comparison are separate consumers. Sources: backend/document_scanner.py:926-970 (parse_pdf_document); backend/document_scanner.py:111-120 (ocr_image); src/pages/SignupPage.jsx:585-697 (validateCogIdentity). [N] No learned OCR-confidence threshold or trained identity model is present in these functions.

[S] No Supabase Edge Function matching implementation was found in the inspected repository. Database matching/commitment logic resides in SQL migrations. The latest definitions, not an older same-named migration function, are cited here. [U] Deployed SQL/runtime behavior is outside this read-only local audit.

2. Actual Formulas, Weights, and Similarity Thresholds

[S] Let D(i,j)=min(D(i-1,j)+1, D(i,j-1)+1, D(i-1,j-1)+[a_i!=b_j]), initialized with D(i,0)=i and D(0,j)=j. Insertions, deletions and substitutions each cost one. For nonempty normalized strings, S=1-D(m,n)/max(m,n). S is a decimal in [0,1], not observed accuracy. The Python scalar helpers return 1 for both-empty strings, while SignupPage getLevenshteinSimilarity returns 0 for both empty; their caller missing-value rules therefore matter.

System Component	Implemented Algorithm	Actual Formula	Threshold / provenance	System Decision	Source Code Reference
[S] Student identity/name matching (signup roster)	Custom normalized Levenshtein + token overlap	S=1-d/max(lengths). Match if direct S>=.72 OR sorted S>=.72 OR (same last token AND overlap>=.5).	.72 and .5: hardcoded. Missing/equal names pass this helper.	Different-name same-ID roster can be reported as identity conflict; this helper is not official award authorization.	backend/signup_service.py:485-508 (levenshtein_similarity); backend/signup_service.py:549-562 (is_similar_roster_name)
[S] Browser ROG-to-COR identity comparison	Custom DP + token overlap + 2-factor weighted score	n=max(sorted-name S, token overlap); i=1 only for exact digit-only ID; Q=round(.75i+.25n,4).	ID exact required; when both names exist Q>=.85 AND n>=.70. All hardcoded. Missing-name path skips name gate.	Returns passed/canValidate metadata. Current caller logs failure as inconclusive; it does not reject on this result.	src/pages/SignupPage.jsx:585-697 (validateCogIdentity); src/pages/SignupPage.jsx:981-1093 (scanUploadedDocument)
[S] Fuzzy record comparison	Custom Levenshtein and direct/token-sorted name maximum	n=max(S(nameA,nameB), S(sortTokens(nameA),sortTokens(nameB))).	No global scalar-helper threshold. Branches below define decisions; missing field comparisons return None.	Computes text similarity before exact flags; not restricted to missing/different IDs.	backend/grantor_algorithms.py:32-56 (levenshtein_similarity); backend/grantor_algorithms.py:59-65 (comparable_similarity); backend/grantor_algorithms.py:68-123 (evaluate_scholar_duplicate)
[S] Weighted record linkage	Eight-field available-weight normalized sum	W=sum(w_i*s_i)/sum(w_i), only values not None; W=0 if included weight=0.	Name >=.82 AND W>=.84 for general pair rule. Both hardcoded; exact/contact alternatives can bypass.	Possible duplicate flag, not proof of one student or scholarship commitment.	backend/grantor_algorithms.py:68-123 (evaluate_scholar_duplicate)
[S] Strong identifier/contact pair rules	Exact normalized ID/email and digit-only phone + name	duplicate = exactID OR ((exactEmail OR exactPhone) AND n>=.72) OR (n>=.82 AND W>=.84).	Exact ID has no numeric score floor; .72 hardcoded. Field reason-label threshold .90 is not a duplicate gate.	Same ID can flag despite different names; unequal IDs can still flag through the fuzzy branch.	backend/grantor_algorithms.py:68-123 (evaluate_scholar_duplicate)
[S] Admin duplicate grouping	Greedy grouping using pair flag OR score-only comparison	Join first group if any member: pair.isDuplicate OR pair.score>=T.	T=float(options.get("threshold") or .84): dynamic parameter, default hardcoded .84. Numeric zero falls back.	Score-only grouping may bypass .82 name floor. No built-in probable/uncertain tier or validated T range.	backend/grantor_algorithms.py:206-262 (check_student_table_duplicates)
[S] Legacy/admin linkage helper	Exact ID, otherwise normalized name parts AND shared address	Exact lookup key OR match_name_parts AND match_address. Missing middle initial tolerated.	Exact Boolean rules; no Levenshtein cutoff. Admin lookup takes first suitable fallback.	Relationship/lookup output; not the current authoritative award transaction.	backend/grantor_algorithms.py:326-333 (matches_grantor_scholar_to_student); backend/grantor_algorithms.py:366-396 (match_admin_grantor_students)

System Component	Implemented Algorithm	Actual Formula	Threshold / provenance	System Decision	Source Code Reference
[S] Current roster import identity	Exact explicit IDs; exact canonical profile if either ID absent	Both IDs present: ID equality. Otherwise exact (name, canonical course, normalized year) tuple equality.	No fuzzy threshold. Latest profile normalization from Oct 26 migration; missing profile field prevents match.	Same-cycle active cross-grantor matches block import. No-ID rows stay unlinked and do not authorize an account award.	backend/roster_workflow_service.py:141-146 (_same_roster_identity); supabase/migrations/20261026043502_manual_roster_name_and_course_matching.sql:3-32 (roster_import_profile_key); supabase/migrations/20261026034945_cross_grantor_roster_identity_guard.sql:16-24 (roster_import_matches)
[S] Duplicate scholarship conflict prevention	Stored identity/program/cycle/commitment rules, transaction locks	Exact stored student ID and normalized full name; current cycle; recognized active program/provider; no competing live award/commitment.	No similarity threshold. Current cycle is configured; unknown-cycle active legacy rows require resolution.	Named blocker/exception prevents conflicting assignment/application. Full Admin resolution requires role and reason.	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:69-194 (assign_authoritative_roster_scholarship); supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:447-474 (roster_application_blocker); backend/roster_workflow_service.py:520-542 (resolve_roster_conflict)
[S] COR extraction and validation	pdfplumber + field regex/layout; optional local OCR fallback	Extract labelled identity/course/year/section/GWA/period. Signup compares exact COR ID and current period; title/policy/year checks.	No OCR/Levenshtein confidence threshold. Current cycle and corMode configured. File limit 10 MiB.	Reject invalid title/ID/cycle or missing usable year in signup batch. Profile replacements do not repeat all those checks.	backend/document_scanner.py:926-970 (parse_pdf_document); backend/signup_service.py:172-306 (create_signup_document_batch); backend/student_profile_service.py:885-967 (upload_student_document)
[S] ROG extraction and validation	Table -> positioned-word -> text grade extraction + title/period rules	Read printed GWA; flag Final 4/5, INC/UD/OD or Failed remarks. Signup requires prior period unless exempt.	No OCR confidence threshold. Concern tokens hardcoded; prior cycle derived. Browser Q gates are distinct and currently advisory.	Reject wrong title/period/concerns in signup batch. ROG owner check is not enforced there; profile upload principally checks title.	backend/document_scanner.py:458-531 (extract_final_grades_from_pdf_tables); backend/document_scanner.py:534-658 (extract_final_grades_from_pdf_words); backend/document_scanner.py:661-731 (extract_final_grades_from_text); backend/signup_service.py:172-306 (create_signup_document_batch)

Attribute weights and missing values

Field	Raw weight	Rule
Student ID	.32	Hardcoded in evaluate_scholar_duplicate
Full name	.30	Hardcoded in evaluate_scholar_duplicate
Email	.10	Hardcoded in evaluate_scholar_duplicate
Contact number	.08	Hardcoded in evaluate_scholar_duplicate
Course	.08	Hardcoded in evaluate_scholar_duplicate
Year level	.04	Hardcoded in evaluate_scholar_duplicate
City	.04	Hardcoded in evaluate_scholar_duplicate
Province	.04	Hardcoded in evaluate_scholar_duplicate

[S] Raw weights sum to 1.00. Only comparisons whose value is not None enter the denominator. The name expression name_similarity or None excludes a computed zero, whereas zero-valued comparisons in other fields remain. Missing fields renormalize the effective weights; identical names alone can reach W=1.00. Birthdate, school, street and barangay are not weighted fields. No calibration, training, probabilistic interpretation, or experimentally optimized threshold is evidenced in this function.

[S] Token overlap for signup names is |uniqueTokens(A) intersect uniqueTokens(B)| / max(uniqueTokenCount(A),uniqueTokenCount(B)), zero for no tokens. With exact ID=1 and name floor .70, the browser .85 score condition is weaker than the name floor: .75+.25*.70=.925. Missing names take a bypass path despite canValidate=false. [U] No ablation or threshold-tuning dataset was found; do not call these cutoffs experimentally optimal.

3. Existing Evaluation Metrics and Test Evidence

[S] scripts/evaluate-solis-matching.py:evaluate calculates TP, FP, TN, FN, Accuracy, Precision and Recall from expectedDuplicate and the actual backend pair flag. F1 is absent from that existing function. It uses None for zero denominators. The labels are pre-existing synthetic scenario intent, not verified registrar/award-provider evidence. The audit derives F1 from the same recorded counts without modifying that evaluator. Source: scripts/evaluate-solis-matching.py:14-36 (evaluate).

[T] Actual invocation: py -3.12 "docs/ThesisPaper/Algorithm_Metrics_Audit/run_metrics_audit.py". Environment: Windows-11-10.0.26300-SP0; Python 3.12.2. UTC start/end: 2026-10-28T04:28:01.667642+00:00 / 2026-10-28T04:28:22.113337+00:00. External application networking was blocked; internal Windows asyncio socketpairs were allowed. Storage/database boundaries in workflow tests were mocked. No dependency, OCR engine or production data was installed/modified.

Evidence	Input / expected basis	Actual outcome	Metric interpretation / classification
Weighted_Matching_Execution.json	Ten existing expectedDuplicate-labeled synthetic pairs	TP5 FP1 FN0 TN4; existing metrics reproduced	[T] Combined pair result only; author-assigned fixture labels
Matching_Regression_Execution.txt / Results.json	Existing Solis and roster workflow unittest contracts	48 tests; failures 0; errors 0	[T] Regression success, not 48 true positives
Current_Roster_SQL_Execution.json	Seven new synthetic Boolean contract assertions using latest pure functions	7/7 assertions passed in PGlite	[T] Exact canonicalization tests, not labeled conflict accuracy
Document_Reexecution/Existing_Test_Execution.txt	Existing isolated document/signup tests	33 tests, 0 failures/errors	[T] Some parser calls are mocked; pass rate is not extraction accuracy
Document_Reexecution/Field_Comparisons.csv	Three existing TEST PDFs, independent visible-field transcription	COR16/16; ROG9/9 present values	[T] Tiny synthetic text-PDF field measurement only
Document_Reexecution/Evaluation_Results.json	Fifteen real-parser/in-memory workflow scenarios	9 expected outcomes; 4 limitations; 2 OCR blocks	[T] Not a 15/15 success claim; not all registration/HTTP/database paths
test-results/solis-system-cases.json	Previously recorded isolated transaction assertions	27 recorded Pass entries inspected, not rerun in this audit	[U] Historical artifact, not fresh current SQL transaction acceptance

[T] Full matching/roster test identifiers, source lines, original assertion/input code, actual status and ground-truth limitations are retained in Matching_Regression_Results.json. The unittest console contains the executed method names. Every evaluation-pair input/label/output is retained in Table C CSV and Weighted_Matching_Execution.json. Every evaluated document field is in the fresh field-comparison CSV. These provide input/expected/actual traceability without exposing production students.

[T] Test definitions and assertions are not the same as independent ground truth. Boundary tests patch similarity values to verify rule logic; they do not estimate real matching performance. The pre-existing dataset includes an empty-record negative and no labeled false-negative example; neither absence of missed matches in ten pairs nor passing lock/role assertions establishes recall=100% for actual students.

4. Precision, Recall, F1-score, and Accuracy Evaluation Tables

[P] Positive identity class: both records refer to the same independently established student. TP=correct predicted match; FP=predicted match for different identities; FN=missed same identity; TN=correct nonmatch. For the executed fixture, replace independent real identity with its pre-existing expectedDuplicate label and disclose that limitation. Do not relabel a denied award as a positive identity match.

[P] Precision=TP/(TP+FP); Recall=TP/(TP+FN); F1=2PR/(P+R); Accuracy=(TP+TN)/(TP+FP+FN+TN). An equivalent count-form F1 is 2TP/(2TP+FP+FN). The audit returns Not defined/None for denominator zero, not a fabricated zero or 100%. Count-form F1 can be 0 when errors exist even if P or R is undefined; the all-empty case remains undefined. Report the convention rather than silently excluding such cases.

[T] Existing evaluate([]) produced None for precision, recall and accuracy. An all-negative/no-predicted-positive fixture produced undefined precision/recall rather than invented percentages. These checks are recorded in Weighted_Matching_Execution.json. For this sample, Precision=5/6, Recall=5/5, F1=10/11 and Accuracy=9/10.

Table B. Fuzzy Matching and Duplicate Detection Performance Evaluation

Evaluation Component	TP	FP	FN	TN	Precision (%)	Recall (%)	F1-Score (%)	Accuracy (%)
Levenshtein-based name classifier alone [U]	-	-	-	-	-	-	-	-
Backend weighted linkage + exact/contact rules [T]; ten synthetic fixture labels only	5	1	0	4	83.33	100.00	90.91	90.00
Duplicate scholarship conflict detection [U]	-	-	-	-	-	-	-	-
Frontend ROG identity comparator [U]; advisory result	-	-	-	-	-	-	-	-

[T/U] Only the combined backend pair mechanism has a reproduced labeled fixture result. Its 83.33% precision, 100% recall, 90.91% F1 and 90% accuracy are not standalone Levenshtein, separate weighted-only ablation, browser ROG, or scholarship-conflict results. A dash means Not yet evaluated for a suitable independent classification dataset, not zero. Independent real-student validation remains pending for every mechanism.

Table C. Actual Matching Decision Evaluation

[T] Inputs below are the existing synthetic fixture records, not real student information. Similarities are printed as decimals. Ground truth is the fixture author's pre-existing positive/negative label, independent of the newly executed prediction, but not independently adjudicated identity. Exact full values and field contributions are also in the CSV/JSON. A false-positive label is not converted into a pass just because the function follows its configured rules.

Test Case ID	Record A	Record B	Verified Ground Truth	Calculated Similarity	Applied Threshold	System Decision	Classification
MATCH-001	{"studentId":"SYN-001","fullName":"Fixture Scholar One"}	{"studentId":"SYN001","fullName":"Fixture Scholar One"}	Same [synthetic label]	name=1.000000; W=1.000000	Exact normalized Student ID	Match	TP

Test Case ID	Record A	Record B	Verified Ground Truth	Calculated Similarity	Applied Threshold	System Decision	Classification
MATCH-002	{"studentId":"SYN-002","fullName":"Fixture Scholar Two"}	{"studentId":"SYN002","fullName":"Different Recorded Name"}	Same [synthetic label]	name=0.260870; W=0.642356	Exact normalized Student ID	Match	TP
MATCH-003	{"fullName":"Sample Student Three"}	{"fullName":"Three Sample Student"}	Same [synthetic label]	name=1.000000; W=1.000000	Name >= .82 AND weighted score >= .84	Match	TP
MATCH-004	{"fullName":"Example Student Four","email":"four@example.test"}	{"fullName":"Example Studen Four","email":"four@example.test"}	Same [synthetic label]	name=0.950000; W=0.962500	Exact contact + name >= .72	Match	TP
MATCH-005	{"fullName":"Fixture Scholar Five","contactNumber":"09170000005"}	{"fullName":"Fixture Scholar Fiv","contactNumber":"09170000005"}	Same [synthetic label]	name=0.950000; W=0.960526	Exact contact + name >= .72	Match	TP
MATCH-006	{"fullName":"Ana Santos","email":"household@example.test"}	{"fullName":"Robert Cruz","email":"household@example.test"}	Nonmatch [synthetic label]	name=0.090909; W=0.318182	No duplicate rule met	Nonmatch	TN
MATCH-007	{"fullName":"Ana Santos","contactNumber":"09170000000"}	{"fullName":"Robert Cruz","contactNumber":"09170000000"}	Nonmatch [synthetic label]	name=0.090909; W=0.282297	No duplicate rule met	Nonmatch	TN
MATCH-008	{"studentId":"SYN-101","fullName":"Ana Santos"}	{"studentId":"OTHER-999","fullName":"Robert Cruz"}	Nonmatch [synthetic label]	name=0.090909; W=0.043988	No duplicate rule met	Nonmatch	TN
MATCH-009	{}	{}	Nonmatch [synthetic label]	name=0.000000; W=0.000000	No duplicate rule met	Nonmatch	TN
MATCH-010	{"studentId":"SYN-201","fullName":"Shared Common Name","course":"BSIT","year":"4","city":"Baliwag","province":"Bulacan"}	{"studentId":"SYN-202","fullName":"Shared Common Name","course":"BSIT","year":"4","city":"Baliwag","province":"Bulacan"}	Nonmatch [synthetic label]	name=1.000000; W=0.934959	Name >= .82 AND weighted score >= .84	Match	FP

[T] MATCH-001 through MATCH-005 are TPs; MATCH-006 through MATCH-009 are TNs; MATCH-010 is the single FP. No FN occurred in this fixture. MATCH-009 consists of two empty dictionaries: its negative software label is not proof of two distinct real people. This test-design limitation must remain visible in any thesis use of the aggregate.

[P] Separate scholarship-conflict gold labels must mean a new operation would create a prohibited award/commitment for the same student and applicable cycle. TP=correct conflict block/referral; FP=incorrect conflict block/referral; FN=unblocked prohibited conflict; TN=correctly permitted nonconflicting action. Invalid file/role/program errors should be a separate error category, not automatically conflicts. Label independent authoritative awards, dates/cycles and applicable policy before prediction. Existing transaction test assertions alone do not supply representative conflict ground truth.

5. COR and ROG Extraction Evaluation

[S] Document extraction is separate from identity linkage. pdfplumber reads embedded PDF text/layout; local pdf2image/Tesseract OCR is attempted only when the combined PDF text is empty. ROG final grades use table cells, word-position columns and text fallback. Name, ID, course, year, section, printed GWA, academic year and semester are returned. MiddleName is currently blank; no learned confidence is returned. Sources: backend/document_scanner.py:123-137 (extract_pdf_text); backend/document_scanner.py:926-970 (parse_pdf_document); backend/document_scanner.py:887-923 (parse_document).

[S] COR/ROG routes require PDF files; generic image helpers exist, but a PNG COR/ROG is unsupported by policy. Scanned PDFs are supported in design only with runtime dependencies. OCR text extraction does not itself invoke Levenshtein. The browser ROG-to-COR comparator is a later advisory check; signup roster-name comparison is a different later identity path.

[T] Fresh run of three existing visibly synthetic TEST PDFs reproduced 25/25 selected present values. Reference values came from visible labels, not the evaluated parser. ID equality was exact after trimming; canonical names/course/semester used predeclared trimmed case-insensitive matching. Absent middle names, sections and COR GWA were excluded, not counted as correct blanks. Name/full-name fields are correlated. Source file hashes and all expected/actual values are retained in the fresh Document_Reexecution folder.

Table D. COR and ROG Data Extraction Evaluation

Document Type	Evaluated Field	Extraction/Validation Method	Correct Results	Evaluated Total	Evaluation Metric	Measured Result
COR	academicYear	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	2	2	Field-value accuracy [T]	2/2 = 100.00% (synthetic only)
COR	course	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	2	2	Field-value accuracy [T]	2/2 = 100.00% (synthetic only)
COR	firstName	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	2	2	Field-value accuracy [T]	2/2 = 100.00% (synthetic only)
COR	fullName	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	2	2	Field-value accuracy [T]	2/2 = 100.00% (synthetic only)
COR	lastName	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	2	2	Field-value accuracy [T]	2/2 = 100.00% (synthetic only)
COR	semester	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	2	2	Field-value accuracy [T]	2/2 = 100.00% (synthetic only)
COR	studentId	Exact trimmed Student ID	2	2	Field-value accuracy [T]	2/2 = 100.00% (synthetic only)
COR	year	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	2	2	Field-value accuracy [T]	2/2 = 100.00% (synthetic only)
ROG	academicYear	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)
ROG	course	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)
ROG	firstName	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)
ROG	fullName	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)
ROG	gwa	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)
ROG	lastName	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)
ROG	semester	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)
ROG	studentId	Exact trimmed Student ID	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)

Document Type	Evaluated Field	Extraction/Validation Method	Correct Results	Evaluated Total	Evaluation Metric	Measured Result
ROG	year	Predeclared canonical string, trimmed/case-insensitive; parser labels/layout/regex	1	1	Field-value accuracy [T]	1/1 = 100.00% (synthetic only)
COR/ROG	OCR page transcription	No operational OCR run/independent OCR reference alignment	-	-	CER/WER [U]	Not yet evaluated
COR/ROG	Field detection (presence or entity values)	No predeclared detection annotation/matching protocol executed	-	-	Precision/Recall/F1 [U]	Not yet evaluated

[T] COR aggregate=16/16=100.00%; ROG aggregate=9/9=100.00%, only on two simple CORs and one ROG. This is neither OCR accuracy nor Levenshtein accuracy, and it does not establish performance across authentic documents or grade tables. The executed ROG fixture contains printed GWA but no representative multi-row grade table benchmark.

[T] Validation outcomes are separate: wrong COR owner, missing detected/declared year, wrong COR/ROG period, malformed PDF and COR image format were rejected in applicable isolated cases. Four limitations persisted: profile COR allowed another ID through to the mocked approval collaborator; profile ROG allowed an incorrect period; signup batch preparation accepted absent COR course; signup batch preparation accepted a different ROG ID. These are isolated function results, not final account creation or persisted production approvals. Two OCR probes were dependency-blocked; Poppler was absent and the configured Tesseract path did not exist despite a misleading health flag.

[S] Correction options: signup can autofill and allow editing displayed names, year and GWA, and can accept a declared year when detected year is absent or flag disagreement for review. Editing a form field does not repair the original PDF or prove the server validated all underlying scan values. Rejected academic documents require a corrected upload; immutable versions and reviews are distinct. Sources: src/pages/SignupPage.jsx:708-712, 1998-2005, 2250-2298, 2488; backend/signup_service.py:172-306 (create_signup_document_batch); backend/student_profile_service.py:885-967 (upload_student_document).

[P] Appropriate current metric is present-reference field-value accuracy: correct applicable field values / all applicable reference fields. For a future field-presence study, define each field slot's present/absent truth; a present but wrong value can be a presence TP yet still fail value accuracy. For value/entity recovery, define one-to-one reference/prediction matching: a wrong returned value can contribute an FP and a missed gold value an FN. Do not mix these scoring protocols.

[P] CER=(character substitutions+deletions+insertions)/reference character count; WER=(word substitutions+deletions+insertions)/reference word count, with edit alignment and a prespecified whitespace/tokenization protocol. Reference transcription must be independently prepared from authorized document images, not the parser's output. Rates may exceed 100% with many insertions. Empty reference denominators are undefined or must be reported as a separate insertion-only category under a predeclared protocol. [N/U] No CER/WER or OCR/field-detection P/R/F1 implementation/evaluation was found or executed here.

6. Step-by-Step Algorithm Example

[T] Observed example MATCH-004 was selected from the existing fixture, not invented for the report. A={fullName: Example Student Four, email: four@example.test}; B={fullName: Example Studen Four, email: four@example.test}. The existing fixture labels it a same-identity synthetic pair.

[S/T] Normalization yields "example student four" (20 characters) and "example studen four" (19). Email identifier normalization yields "fourexampletest" on both sides because punctuation is removed. No Student ID is supplied. The actual scalar helper returned 0.95 for direct and token-sorted names.

[T] One deletion of the t in student transforms the normalized A name into B; edit distance=1. Therefore S=1-1/max(20,19)=1-1/20=.95. The dynamic program permits insertion/deletion/substitution of unit cost; no cost or threshold is inferred from the test result.

[S/T] Only name weight .30 and email weight .10 are available. W=(.30*.95+.10*1)/(.30+.10)=(.285+.10)/.40=.9625. Missing ID, phone, course, year, city and province contribute neither similarity nor weight in this pair.

[T] The actual exactEmail flag is true and name=.95 >= .72, so the strong-contact branch flags a duplicate. The general fuzzy branch also passes because .95 >= .82 AND .9625 >= .84. The observed isDuplicate value is true. No scholarship approval, current award, OCR confidence or 96.25% identity probability follows from that score.

[T] The pre-existing synthetic reference label is true; prediction=true, so this fixture contributes one TP. One such example cannot establish precision/recall; those metrics require labeled positive AND negative cases across the collection. No independent registrar verification of a real student was performed.

[T] Counterexample MATCH-010 has different synthetic IDs SYN-201/SYN-202 but the same name, course, year, city and province. ID similarity=5/6; all five other included comparisons=1. Included weight=.82, giving W=(.32*(5/6)+.50)/.82=0.934959349593. Name similarity=1, so the fuzzy .82/.84 rule flags despite different IDs. The synthetic label is nonmatch, making this an FP, not a conflicting official scholarship award.

[T] The collection has TP5, FP1, FN0, TN4: Precision=5/(5+1)=83.33%; Recall=5/(5+0)=100%; F1=2*5/(2*5+1+0)=90.91%; Accuracy=(5+4)/10=90%. These decimal/percentage conversions are evaluation metrics, unlike the internal similarity .95 or weighted score .9625.

7. Proposed Chapter 3 Methodology

[P] Suggested thesis subsection: "Evaluation of Duplicate-Record Matching and Document Extraction." The paragraphs below distinguish executed procedures (past tense) from additional work (future). Adapt table/section numbers without converting fixture evidence into a real-student study.

A controlled synthetic evaluation was conducted to examine the combined duplicate-record decision implemented by evaluate_scholar_duplicate. The evaluation used the ten record pairs and Boolean expectedDuplicate labels already defined in the project fixture. These labels represented predeclared synthetic scenario intent and were not derived from the newly generated predictions. They did not constitute independent identity verification of actual students. The existing normalization, custom Levenshtein similarity, field weights and decision rules were executed without changing their implementation.

Levenshtein similarity was calculated as one minus the unit-cost edit distance divided by the longer normalized string length. Name comparison used the greater of direct-order and token-sorted similarity. Available attribute comparisons were combined as the weighted similarity sum divided by the included weight sum. The pair decision accepted exact normalized Student ID; exact email/contact with at least .72 name similarity; or at least .82 name similarity together with at least .84 weighted similarity. The administrative grouping rule and browser ROG diagnostic were treated as separate, unevaluated classifiers rather than assigned the pair classifier's metrics.

Predictions were compared with the pre-existing synthetic labels to count true positives, false positives, false negatives and true negatives. Precision measured the proportion of predicted matches consistent with those labels; recall measured the proportion of labeled matches detected. F1-score summarized their balance, while accuracy summarized all label agreements. Undefined denominators were reported as not defined. The interpretation was restricted to this small synthetic fixture set, including its incomplete-record negative case; no population-level identity-detection effectiveness was inferred.

COR/ROG processing was evaluated separately using two existing synthetic COR PDFs and one synthetic ROG PDF with independently transcribed visible reference values. Present-field value accuracy was measured with explicit numerators and denominators; document-validation outcomes and OCR dependency failures were recorded separately. Regression assertions and local exact-profile SQL tests were reported as software-behavior evidence rather than converted into identity or scholarship-conflict confusion counts.

A subsequent independently adjudicated evaluation is required for representative student identities and academic-cycle conflicts. It should predeclare the positive class, sampling, entity-level train/test separation where relevant, label-review process, thresholds, error categories and missing-value policy. Scholarship-conflict labels must be established from authoritative award/cycle evidence separately from identity labels. OCR CER/WER and field-detection precision/recall require additional annotations and an operational authorized evaluation environment.

8. Proposed Chapter 4 Results and Discussion

[P/T] Suggested title: "Synthetic Duplicate-Record Matching Results and Evidence Limitations." Table B is the measured fixture result; Table C provides its individual decisions. Do not add an unperformed standalone Levenshtein, ablation, live-conflict or OCR accuracy result.

The combined backend matching mechanism produced five true-positive, one false-positive, zero false-negative and four true-negative decisions against ten pre-existing synthetic labels. Precision was 83.33% (5/6), recall was 100.00% (5/5), F1-score was 90.91% (10/11), and accuracy was 90.00% (9/10). These measurements described only the selected synthetic fixture; the labels were not independently adjudicated real-student identities. The result should not be generalized to all student records or reported as standalone Levenshtein or scholarship-conflict performance.

The false-positive case contained different Student IDs with the same name, course, year and geographic fields. The combined score exceeded the implemented fuzzy threshold despite the different IDs. This observation demonstrates that agreement in multiple common fields can produce a possible-duplicate flag for a labeled nonmatch. No false-negative case occurred in this small collection, but this does not establish that real duplicates would never be missed. The test mix, empty-record negative and limited variations constrain interpretation.

The separate document evaluation obtained 16 correct COR field comparisons out of 16 and nine correct ROG comparisons out of nine across three synthetic text PDFs. This limited 100% field result was not an OCR accuracy measurement or evidence that every submission was correctly validated. Four owner/period/completeness limitations remained in isolated workflow scenarios, while two OCR probes were blocked by missing local dependencies. These outcomes were retained separately from successful regression assertions.

The inspected system distinguishes potential duplicate identities from prohibited scholarship commitments. Official assignment uses separate identity, program, cycle and commitment rules. Although local contract and previously recorded transaction evidence support selected behavior, a representative independently labeled scholarship-conflict evaluation was not available; its precision, recall and F1-score remain not yet evaluated. Representative student-record/OCR evaluation and threshold-sensitivity analysis are also required before broader effectiveness claims.

[P] Table explanations: Table A describes implementation formulas and where thresholds originate, not performance. Table B compares prediction/label agreement for evaluated mechanisms only. Table C exposes each underlying synthetic pair and branch decision so the aggregate can be checked. Table D isolates document field recovery from matching and acceptance. The worked example explains one actual computation, not an independent estimate of precision or recall.

9. Research Objective-to-System Traceability

Research Objective	Actual System Function	Algorithm or Method	Evaluation Metric	Source Evidence	Result
Student-record validation	Signup roster identity + import/profile matching	Exact checks / custom similarities / canonical profiles	Contract tests; stage-specific P/R pending	backend/signup_service.py:549-562 (is_similar_roster_name); backend/roster_workflow_service.py:141-146 (_same_roster_identity)	[T] Relevant regressions passed; [U] no separate labeled-stage metric
Duplicate identity detection	evaluate_scholar_duplicate	Combined exact/contact + weighted Levenshtein rules	P=83.33%; R=100%; F1=90.91%; A=90% on ten synthetic cases	backend/grantor_algorithms.py:68-123 (evaluate_scholar_duplicate); scripts/evaluate-solis-matching.py:14-36 (evaluate)	[T] TP5/FP1/FN0/TN4; [U] no independent population accuracy
Scholarship conflict prevention	assign_authoritative_roster_scholarship / roster_application_blocker	Exact identity, recognized program, cycle and commitment guards	Independent conflict P/R/F1 pending	supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:69-194 (assign_authoritative_roster_scholarship); supabase/migrations/20261023070747_solis_cycle_aware_roster_reconciliation.sql:447-474 (roster_application_blocker)	[T] Source/contract evidence; [U] no representative labeled conflict dataset
Current no-ID import checks	roster_import_matches / _same_roster_identity	Exact profile canonicalization when either ID absent	Seven SQL Boolean contract assertions	supabase/migrations/20261026034945_cross_grantor_roster_identity_guard.sql:16-24 (roster_import_matches); supabase/migrations/20261026043502_manual_roster_name_and_course_matching.sql:3-32 (roster_import_profile_key)	[T] 7/7 local PGlite assertions; not a confusion matrix
COR/ROG processing	parse_pdf_document + signup/profile workflow	Embedded text/layout, regex, optional OCR; independent document policy	COR16/16; ROG9/9 present fields	backend/document_scanner.py:926-970 (parse_pdf_document); backend/signup_service.py:172-306 (create_signup_document_batch); backend/student_profile_service.py:885-967 (upload_student_document)	[T] Three synthetic PDFs; four limitations and two OCR blocks retained

[S/T] The result of a fuzzy pair helper is not authoritative approval. Current grantor create/import paths call server preview/commit; older fuzzy import branches remain below return/throw paths, while scholar editing still invokes the pair helper. Current October 26 imports additionally support exact profile fallback for missing IDs. Research wording must describe the actual evaluated call path, not reuse an older diagram or assume every helper is an award guard.

10. Missing Evidence and Actions Required

[U/P] Obtain authorized, independently labeled representative student pairs. Establish same/different identity through institutional identifiers/records or reviewed synthetic identity construction; document two reviewers and adjudication of disagreement. Do not let the prediction algorithm assign its own truth.

[P] Include spelling changes, reversed names, accents, shared/common names, missing/contact fields, near-but-distinct IDs, genuinely different people sharing a household, and plausible missed-duplicate positives. Prespecify treatment of invalid empty records. Report every inclusion/exclusion, class balance and repeated-student dependence.

[U/P] Evaluate the backend pair decision, admin grouping, signup roster-name rule and frontend diagnostic only for their own implemented classifications. Grouping may require pairwise/link-level or cluster-level scoring with a declared protocol. No separate standalone Levenshtein metric should be inferred from a combined helper result.

[U/P] Build a distinct labeled scholarship-conflict set with verified current/historical awards, completions, cancelled/terminated commitments, pending applications and uncertain legacy cycles. Resolve uncertainty through authorized review rather than forcing a binary label. Report invalid-input/authorization failures separately.

[N/P] Persist F1 in a future evaluator only through a separately approved code change. For this audit it is an independently derived documentation metric. Keep undefined denominator conventions explicit, and report confusion counts alongside all percentages.

[U/P] Validate thresholds on held-out labeled records before tuning. Investigate the common-name FP and possible near-ID bias. Any sensitivity sweep is proposed, not executed here; do not claim the hardcoded cutoffs are optimal. Protect an untouched final evaluation split from tuning leakage.

[T/P] Fix and regression-test the four reproduced academic-document gaps only in a separately authorized implementation task. This audit intentionally left them unchanged. A failed advisory browser comparison is not a server-side identity guard.

[U/P] Establish a separately approved OCR environment with actual executable checks and independent image transcriptions. Keep unreadable/failed scans in the study accounting. Do not count text-only PDF measurements as OCR or claim CER/WER without transcription/alignment evidence.

[U/P] Measure latency/throughput and realistic candidate volumes if the manuscript claims performance effectiveness. The current audit records execution environment/times for reproducibility, not a benchmark or scaling guarantee.

[U/P] Revalidate live deployment, privileged function grants, role enforcement, database transactions and private Storage only under a separate authorized staging/live verification. Ephemeral PGlite and mocked Python workflows do not establish production acceptance or deployed migration state.

[P] Add the supplied Chapter 3/4 wording and Tables A-D to the manuscript manually. Preserve the synthetic-only qualification, the FP, missing independent evaluation and document limitations. This audit did not edit the manuscript or mark the panel revision fully closed.

[T] Reproduce this audit from the repository root with the recorded Python command and the Node command in Reproduce.md. Existing source/manuscript files are hash-checked unchanged; raw logs, exact pair inputs and expected/actual fields are included. No production documents or credentials appear in the evidence.